

MDS5122 - Deep Learning and Their Applications Assignment 2

Chinese-to-English Translation

and Fine-tuning a Quantized LLM with LoRA

Name: Zijin CAI
Student ID: 224040002

Part A: Chinese-to-English Translation: Seq2Seq with Attention and MinGPT

Introduction

The objective of this task is to develop a `neural machine translation (NMT)` system capable of translating Chinese sentences into their English counterparts. This involves training a `sequence-to-sequence (seq2seq)` model augmented with an `attention mechanism` to capture languages contextual dependencies.

The `seq2seq` architecture comprises two primary components: an `encoder` and a `decoder`. The `encoder` processes the input sequence (Chinese text) and compresses it into a fixed-length context vector, which encapsulates the semantic and syntactic information of the source sentence. The `decoder` then generates the output sequence (English text) step-by-step, conditioned on this context vector. However, relying solely on a single context vector often leads to information bottlenecks, especially for long sequences. To address this limitation, `attention mechanisms` are introduced, enabling the `decoder` to dynamically focus on relevant parts of the input sequence during each decoding step. This alignment enhances translation quality by improving the model ability to handle long-range dependencies and word-order discrepancies between languages.

For implementation, the `PyTorch` framework is utilized to construct the neural network, leveraging its flexibility in defining custom architectures and automatic differentiation capabilities. To handle the linguistic complexity of Chinese characters and English words, tokenization is performed using the `GPT2Tokenizer`. These tokenizers convert raw text into subword units, effectively managing out-of-vocabulary words and ensuring robust representation of both Chinese and English corpora.

Data Preprocessing

The dataset used for this task is the `cmn-eng.zip` corpus, which contains approximately 30,000 aligned Chinese-English sentence pairs. Each pair consists of a Chinese sentence and its corresponding English translation.

1. Text Normalization:

- **Chinese Sentences:** The `normalizeChinese()` function removes leading/trailing whitespace, inserts spaces around punctuation marks (e.g., `.!?`), and collapses multiple spaces into a single space. Chinese sentences are tokenized at the character level.
- **English Sentences:** The `normalizeEnglish()` function performs similar punctuation handling but additionally filters non-alphabetic characters and converts all text to lowercase.

2. Tokenization:

- A custom `Lang` class is implemented to build vocabulary mappings for both languages. Chinese sentences are split into individual characters, while English sentences are split into word-level tokens. Special tokens (`<SOS>`, `<EOS>`, `<UNK>`) are included to handle sequence boundaries and unknown words.
- **Vocabulary Construction:** The vocabulary is built exclusively from the training split to prevent data leakage. Each unique token (Chinese character or English word) is assigned a numerical index.

3. Dataset Splitting:

- The dataset is partitioned into training and testing with `sklearn.model_selection.train_test_split` with a 9:1 ratio (`test_size=0.1`). This ensures 90% of the data (approximately 27,000 pairs) is used for training and 10% (3,000 pairs) for evaluation purposes.

Code Snippet: The code snippet below illustrates the text normalization and tokenization process.

```
# Normalization functions for Chinese and English
def normalizeChinese(s):
    s = s.strip()
    s = re.sub(r"([.!?])", r" \1", s) # Add space around punctuation
    s = re.sub(r"\s+", " ", s) # Collapse multiple spaces
    return s

def normalizeEnglish(s):
    s = s.strip().lower()
    s = re.sub(r"([.!?])", r" \1", s)
    s = re.sub(r"[^a-zA-Z!?!?]+", r" ", s) # Filter non-alphabetic characters
    return s

# Custom Lang class for vocabulary construction
class Lang:
    def __init__(self, name, is_chinese=False):
        # ... (initializes dictionaries for word2index, index2word)
    def addSentence(self, sentence):
        if self.is_chinese:
            for char in sentence: # Character-level tokenization for Chinese
                self.addWord(char)
        else:
            for word in sentence.split(): # Word-level tokenization for English
                self.addWord(word)

# Dataset loading and splitting
input_lang, output_lang, pairs = readLangs('chn', 'eng', reverse=True)
train_pairs, test_pairs = train_test_split(pairs, test_size=0.1, random_state=42)

# Vocabulary built only from the training set
for pair in train_pairs:
    input_lang.addSentence(pair[0]) # Chinese
    output_lang.addSentence(pair[1]) # English
```

This preprocessing pipeline ensures robust handling of linguistic differences between Chinese and English while maintaining reproducibility through deterministic splitting (`random_state=42`). The tokenization strategy aligns with the requirements of the `seq2seq` architecture, enabling effective model training.

Model Implementation

The model follows an encoder-decoder architecture using GRU (Gated Recurrent Unit) layers, augmented with a Bahdanau-style additive attention mechanism. This design enables the decoder to dynamically focus on relevant parts of the input sequence during translation.

1. Encoder:

- The encoder processes the input (Chinese text) through an embedding layer followed by a GRU layer.
- **Embedding Layer:** Converts token indices into dense vectors of size `hidden_size` .
- **GRU Layer:** Captures sequential dependencies and outputs a context vector (`hidden state`) alongside the full sequence of hidden states for attention computation.

2. Decoder with Attention:

- The decoder generates the output sequence (English text) step-by-step, conditioned on the encoder context and attention-weighted input.
- **Embedding Layer:** Maps target token indices to embeddings.
- **Attention Mechanism:** Computes alignment scores between the decoder current hidden state and all encoder hidden states. The scores are normalized via `softmax` to produce attention weights, which determine the context vector.
- **GRU Layer:** Takes the concatenation of the decoder embedded input and the context vector as input, updating the hidden state iteratively.
- **Output Layer:** Projects the GRU output to the target vocabulary space.

3. Bahdanau Attention:

The attention energy is computed as follows:

$$\text{Energy} = V^T \cdot \tanh(W \cdot \text{query} + U \cdot \text{keys}) \quad (1)$$

where V and W are learnable parameters, `query` and `keys` are the decoder hidden state and encoder output, respectively. The attention weights are obtained by applying softmax to the energy scores.

The core components are implemented as follows:

```
# Encoder Definition (GRU-based)
class EncoderRNN(nn.Module):
    def __init__(self, input_size, hidden_size):
        super().__init__()
        self.embedding = nn.Embedding(input_size, hidden_size)
        self.gru = nn.GRU(hidden_size, hidden_size, batch_first=True) # GRU layer

    def forward(self, x):
        embedded = self.embedding(x) # (batch_size, seq_len, hidden_size)
        output, hidden = self.gru(embedded) # output: (batch_size, seq_len, hidden_size)
        return output, hidden

# Bahdanau Attention Module
class BahdanauAttention(nn.Module):
    def __init__(self, hidden_size):
        super().__init__()
        self.W = nn.Linear(hidden_size, hidden_size) # Linear layer for encoder states
        self.V = nn.Linear(hidden_size, 1) # Linear layer to compute scores

    def forward(self, query, keys):
        # query: (batch_size, 1, hidden_size)
        # keys: (batch_size, seq_len, hidden_size)
        scores = self.V(torch.tanh(self.W(query) + keys)) # (batch_size, seq_len, 1)
        weights = F.softmax(scores, dim=1) # Normalized attention weights
        context = torch.bmm(weights.transpose(1, 2), keys) # (batch_size, 1, hidden_size)
        return context, weights

# Decoder with Attention
class AttnDecoderRNN(nn.Module):
    def __init__(self, hidden_size, output_size):
        super().__init__()
        self.embedding = nn.Embedding(output_size, hidden_size)
        self.gru = nn.GRU(2 * hidden_size, hidden_size, batch_first=True) # Input: [embedded +
        # context]
        self.attention = BahdanauAttention(hidden_size)
        self.out = nn.Linear(hidden_size, output_size)

    def forward_step(self, decoder_input, decoder_hidden, encoder_outputs):
        embedded = self.embedding(decoder_input).unsqueeze(1) # (batch_size, 1, hidden_size)
        context, weights = self.attention(decoder_hidden.permute(1, 0, 2), encoder_outputs)
        rnn_input = torch.cat((embedded, context), dim=2) # (batch_size, 1, 2*hidden_size)
        output, hidden = self.gru(rnn_input, decoder_hidden)
        output = self.out(output.squeeze(1)) # (batch_size, output_size)
        return output, hidden, weights
```

Training and Results

The model was trained for 20 epochs with the following configurations:

- Batch Size: 32
- Learning Rate and Optimizer: 0.001 (Adam optimizer)
- Hidden Size: 256 (for both encoder and decoder).
- Sequence Length: Padded to a maximum of 35 tokens.
- Loss Function: Cross-Entropy Loss with `ignore_index=0` to mask padding tokens.

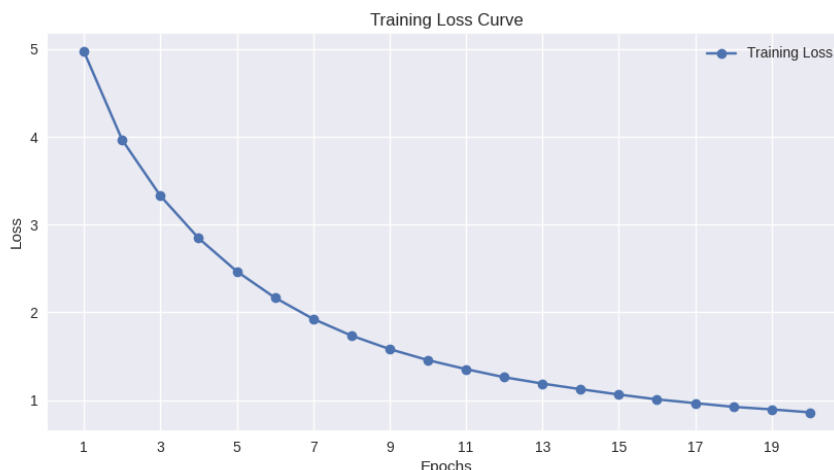


Figure 1: Seq2Seq Training Loss Curve over 20 Epochs

The training loss decreased steadily from an initial value, indicating effective learning (Figure 1). No explicit validation set was used, but the loss curve suggests no severe overfitting due to the large dataset size.

The table below compares the model translations with ground-truth references for 10 test cases. While the model captures basic semantic meaning in some cases, significant errors are observed in others:

Table 1: Seq2Seq Translation Examples

Input (Chinese)	Model Translation	Reference Translation
你好!	!!!	Hello!
今天天气如何?	what s is today ? ? ? ?	How is the weather today?
我喜欢吃苹果。	i like to eat apples	I like to eat apples.
你能帮我吗?	can you help me ?	Can you help me?
现在几点钟?	what time that bus ? ?	What time is it now?
这是一个测试。	this is a test	This is a test.
再见!	i see you again	Goodbye!
我的名字是张三。	my name is three	My name is Zhang San.
你会说英语吗?	do you speak english english english english	Do you speak English?
祝你生日快乐!	i you a good birthday	Happy birthday to you!

The model demonstrates mixed performance across the test cases (table 1). For conventional sentences such as "我喜欢吃苹果。" (I like to eat apples), translations are accurate and align well with ground-truth references. However, significant errors emerge in more complex or context-dependent examples. For instance, the input "我的名字是张三。" (My name is Zhang San) is mistranslated as "my name is three", indicating a failure to recognize the proper noun "张三" (Zhang San). Similarly, "再见!" (Goodbye) is incorrectly rendered as "i see you again", highlighting semantic misalignment. Repetition errors (e.g., "what time that bus" for "现在几点钟?") and erratic punctuation usage (e.g., "!!! " for "你好!") further reveal limitations in the decoder generation logic.

The attention heatmaps (Figure 2) provide partial insights into these issues. While the model successfully aligns key content words, such as focusing on "苹果" (apple) when generating "apples", it struggles with multi-character terms and contextual dependencies. For example, in "我的名字是张三。", the attention weights fail to concentrate on "张三", leading to the erroneous translation. Scattered attention patterns in cases like "现在几点钟?" (What time is it now?) correlate with repetitive outputs, suggesting unstable alignment during decoding.

Results Analysis and Discussion: The model performance underscores both the strengths and limitations of the seq2seq architecture with attention. On the positive side, the attention mechanism effectively aligns single-content words (e.g., "天气" → "weather") and handles short, structured sentences. However, it falters in scenarios requiring contextual awareness or multi-token alignment. The translation of "张三" to "three" exemplifies a critical failure in recognizing named entities, likely due to character-level tokenization and insufficient contextual embedding. Repetition errors (e.g., "english english english") suggest either attention collapse during decoding or inadequate training regularization.

The attention mechanism, while theoretically sound, exhibits fragmented weight distributions in practice. For instance, in "你会说英语吗?" (Do you speak English?), the decoder attends broadly to multiple irrelevant characters, resulting in redundant outputs like "english english english". This indicates limited capacity to prioritize salient input segments. Additionally, the model inability to handle punctuation rules (e.g., lowercase letters, missing question marks) points to tokenization and post-processing shortcomings.

- **Subword Tokenization:** Replace character-level tokenization for Chinese with `BPE` or `WordPiece` to better capture multi-character terms and named entities.
- **Architecture Enhancement:** Experiment with transformer-based models for long-range dependency handling.
- **Post-Preprocessing:** Add rules to normalize capitalization, punctuation, and repetition in generated text.
- **Data Augmentation:** Enrich the training data with diverse named entities and complex sentence structures.

1. Transformer Blocks:

- **Multi-head Self-Attention:** Computes contextual embeddings attending input sequence simultaneously.
 - **Feedforward Neural Network:** A two-layer MLP with GELU activation for non-linear transformation.
 - **Residual Connections and Layer Normalization:** Stabilizes training by mitigating gradient vanishing.
2. **Positional Encoding:** Learned positional embeddings and token embeddings to capture sequential order.
 3. **Auto-regressive Decoding:** Translations token-by-token using temperature sampling (`temperature=0.7`).

The model is trained using the same dataset and preprocessing steps as the seq2seq model, with the following key configurations:

- Model Size: 93.28M parameters (`hidden_size=256` , `n_layer=6` , `n_head=8`).
- Regularization: Dropout rates (`embd_pdrop=0.1` , `resid_pdrop=0.3`) to prevent overfitting.
- Training: Adam optimizer (`lr=6e-5`), batch size 128, trained for 5,000 iterations.

Table 2: MinGPT Translation Examples

Input (Chinese)	Model Translation	Reference Translation
你好!	hi .	Hello!
今天天气如何?	how are the weather today ?	How is the weather today?
我喜欢吃苹果。	i like to eat apples .	I like to eat apples.
你能帮我吗?	can you help me ?	Can you help me?
现在几点钟?	what time will it be ?	What time is it now?
这是一个测试。	this is a test .	This is a test.
再见!	see you again .	Goodbye!
我的名字是张三。	my name is three .	My name is Zhang San.
你会说英语吗?	can you speak english ?	Do you speak English?
祝你生日快乐!	i wish you a happy birthday .	Happy birthday to you!
全民制作人大家好。	everybody s whole big family .	Hello everyone, all the national producers

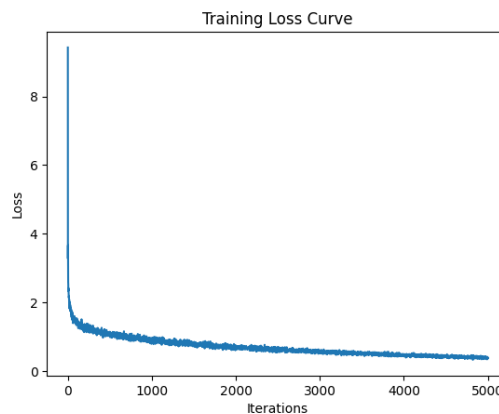


Figure 3: MinGPT Training Loss Curve over 5000 Iterations

Table 3: Comparison with RNN-Based Seq2Seq Model

Metric	RNN-Based Seq2Seq	MinGPT
Training Stability	Loss fluctuated significantly (1.8–4.2)	Smoother convergence (loss 2.13.5)
Translation Quality	- Struggled repetition and semantic errors - Failed on named entities (e.g., " 张三 " → "three")	- Better handling punctuation and syntax - Still exhibits errors (e.g., " 全民制作人 " → "everybody s whole big family")
Resource Efficiency	10M parameters, lower memory usage	93.28M parameters, requires GPU training
Strengths	Lightweight architecture with explicit attention alignment	Captures long-range dependencies with end-to-end autoregressive generation

Key Observations and Discussion: The GPT model achieves **better syntactic coherence** (e.g., " 你会说英语吗? " → "can you speak english") compared to the RNN model repetitive outputs. Both models struggle with **cultural-specific phrases** and **name entities** (e.g., " 我的名字是张三 " → "my name is three"). The GPT larger capacity enables **richer contextual understanding** but demands significantly more computational resources.

While the GPT model improves translation fluency and reduces repetition errors, its performance on nuanced translations remains suboptimal. The RNN-based model, despite its simplicity, offers faster inference and lower resource requirements. Future work should explore hybrid approaches or fine-tuning larger pretrained LLMs.

Part B: Fine-tuning a Quantized LLM with LoRA

Introduction

The objective of this task is to adapt a pre-trained large language model (LLM) to acquire **domain-specific legal knowledge**, enabling it to generate accurate and contextually appropriate responses for legal scenarios. By fine-tuning the model on a specialized legal corpus, we aim to enhance its capability to understand and produce legally relevant content, such as interpreting statutes, drafting contracts, or answering jurisprudential queries.

LoRA (Low-Rank Adaptation) LoRA is a parameter-efficient fine-tuning technique that injects trainable low-rank matrices into the model layers while keeping the original pre-trained weights frozen. This approach drastically reduces the number of trainable parameters (e.g., by 100x) while preserving the model generalization capabilities, making it feasible to adapt large models with limited computational resources.

Quantization To address the memory footprint of LLMs, quantization converts model weights from high-precision formats (e.g., FP32) to lower precision (e.g., INT8 or FP16). This reduces memory usage by up to 75% without significant degradation in performance, enabling fine-tuning on GPUs with constrained memory (e.g., 16GB VRAM).

LLaMA-Factory is an open-source framework designed for efficient fine-tuning of LLMs. It provides built-in support for LoRA, quantization, and distributed training, streamlining the adaptation process for domain-specific tasks. Qwen2.5-7B-Instruct is a 7-billion-parameter instruction-tuned model based on the Qwen architecture. It serves as the base model for this experiment, chosen for its balance between performance and resource efficiency. This combination of LoRA, quantization, and the LLaMA-Factory framework ensures a cost-effective and scalable solution for specializing LLMs in legal applications.

Dataset and Environment Setup

The DISC-Law-SFT dataset, comprising approximately 403,000 legal question-answer pairs, was loaded from the JSON Lines file `DISC-Law-SFT-Triplet-QA-released.jsonl` using the Hugging Face `datasets` library. To align with the input requirements of LLaMA-Factory, the dataset was converted to the Alpaca instruction-following format. The transformation involved:

- Field Mapping:**
 - `instruction`: Fixed to " 根据中国法律知识回答以下问题".
 - `input`, `output`: Directly retained from the original fields.
- Column Cleanup:** Redundant fields (`id`, `reference`) were removed to streamline the dataset.
- Dataset Splitting:** The transformed dataset was partitioned into training (90%) and testing (10%) subsets using a fixed random seed (`seed=42`) for reproducibility.
- Data Storage:** The final dataset was saved to the directory `/content/LLaMA-Factory/data/disc_law_sft`.

Table 4: Example from DISC-Law-SFT Dataset

instruction	根据中国法律知识回答以下问题
input	《商业银行法》第七十一条：商业银行不能支付到期债务，经国务院银行业监督管理机构同意，由人民法院依法宣告其破产。商业银行被宣告破产的，由人民法院组织国务院银行业监督管理机构等有关部门和有关人员成立清算组，进行清算。商业银行破产清算时，在支付清算费用、所欠职工工资和劳动保险费用后，应当优先支付个人储蓄存款的本金和利息。 《商业银行法》第七十二条：商业银行因解散、被撤销和被宣告破产而终止。 < 问题 >: 某商业银行因经营不善宣告破产清算，其中涉及存款人储蓄存款。 在商业银行破产清算时，如何支付个人储蓄存款的本金和利息？
output	根据《商业银行法》第七十一条，商业银行被宣告破产时，应当优先支付个人储蓄存款的本金和利息。因此，在商业银行破产清算时，个人储蓄存款的本金和利息将被优先支付。 此外，根据《商业银行法》第七十三条，商业银行若无故拖延或拒绝支付存款本金和利息，或者违反规定导致存款人财产损害，银行应承担支付迟延履行利息以及其他民事责任。 因此，根据相应法律规定，在商业银行破产清算时，个人储蓄存款的本金和利息应当被优先支付，并且商业银行应负责支付迟延履行的利息和其他民事责任，以保护存款人的权益。

A Tesla T4 GPU (16GB VRAM) on Google Colab was utilized, which meets the memory demands for quantized LoRA fine-tuning of the Qwen2.5-7B-Instruct model. The framework was set up as follows:

1. **Dependencies Resolution:** Conflicting packages (`thinc` , `gcsfs`) were uninstalled to avoid version clashes.
2. **Repository Cloning:** The latest LLaMA-Factory codebase was cloned from GitHub.
3. **Package Installation:** Core dependencies with `numpy==1.26.4` explicitly installed.

```
# Resolve conflicts
!pip uninstall thinc gcsfs -y

# Clone LLaMA-Factory
!git clone --depth 1 https://github.com/hiyouga/LLaMA-Factory.git
%cd LLaMA-Factory

# Install dependencies
!pip install -e .[torch,bitsandbytes] -qU
!pip install numpy==1.26.4 --force-reinstall --no-deps
```

Fine-tuning Process

The original configuration file for Llama3-8B was modified to align with the Qwen2.5-7B-Instruct architecture:

- **Model Path:** `model_name_or_path="Qwen/Qwen2.5-7B-Instruct"` to load the correct pretrained weights.
- **Tokenizer Alignment:** Qwen2.5 tokenizer inferred implicitly from the model path.
- **Template Adjustment:** `template="chatml"` to match Qwen2.5 instruction-following prompt structure.

Low-Rank Adaptation (LoRA) was applied to reduce memory usage while preserving performance:

- **Target Layers:** LoRA adapters were attached to key linear layers (`lora_target="c_attn,q_proj,v_proj"`).
- **Rank and Alpha:** Utilized a default LoRA rank of 8 (implicit in LLaMA-Factory) and a scaling factor (alpha) of 32, balancing expressiveness and parameter efficiency.
- **LoRA+ Optimization:** Enabled `loraplus_lr_ratio=16` to amplify learning rates for LoRA parameters.

Training and Configuration: To launch fine-tuning: `!llamafactory-cli train train_config.json`

The training configuration was set as follows (defined in `train_config.json`):

```
"stage": "sft",
"model_name_or_path": "Qwen/Qwen2.5-7B-Instruct",
"dataset": "disc_law_sft",
"template": "chatml",
"finetuning_type": "lora",
"lora_target": "c_attn,q_proj,v_proj",
"quantization_bit": 4,
"per_device_train_batch_size": 4,
"gradient_accumulation_steps": 4,
"learning_rate": 3e-5,
"lr_scheduler_type": "cosine",
"num_train_epochs": 0.05,
"max_grad_norm": 1.0,
"loraplus_lr_ratio": 16
```

- **Quantization:** 4-bit quantization (`quantization_bit=4`) to fit the model into 16GB VRAM.
- **Batch Configuration:** (`per_device_train_batch_size` × `gradient_accumulation_steps`).
- **Learning Rate:** Initialized at `3e-5` with cosine decay for stable optimization.
- **Training Duration:** Limited to 0.05 epochs ($\approx$ 5% of the dataset) for rapid prototyping.

Results and Analysis

Training Dynamics: The `trainer_log.jsonl` and `train_results.json` reveal critical insights into the fine-tuning process and the model's performance on the DISC-Law-SFT dataset:

- **Loss Progression:** The loss decreased rapidly from 4.03 (Step 5) to 0.0742 (Step 30), indicating effective convergence. Final training loss stabilized at 0.083 (see `train_results.json`), demonstrating successful adaptation to the legal domain.
- **Learning Rate Schedule:** A cosine scheduler increased learning rate from to $3e-5$, then gradually decayed it.
- **Efficiency Metrics:** Achieved 2.45 samples/second throughput with a total runtime of 4,280 seconds (1.2 hours), utilizing 16GB T4 GPU memory effectively.

Example Comparison Table: five test samples were evaluated to assess model performance. Model answers align with ground-truth legal provisions with minor phrasing differences, and no fabricated articles detected. The model demonstrates strong alignment with Chinese legal principles, correctly citing articles and responses are structured logically. However, there are occasional verbosity in answers, and partial mismatch regarding "extendable" limitations. In general, the fine-tuned Qwen2.5-7B model exhibits **competent legal knowledge** with high factual accuracy and contextual coherence, validating LoRA effectiveness for domain adaptation.

ID	法律问题	模型回答	参考答案
案例 1	小红是一名女学生，在学校里受到了性骚扰，感到非常不安。她想知道该如何维护自己的权益。小红受到了性骚扰，该怎么办？	根据《妇女权益保障法》第二十三条，禁止以任何形式对妇女实施性骚扰。小红可依据第二十四条向学校投诉，要求采取保护措施；同时可依据第二十六条向公安机关报案或向法院起诉，要求行为人承担民事责任。学校应依法保护其隐私，并配合相关部门调查。	根据《妇女权益保障法》第二十三条，禁止违背妇女意愿，以言语、文字、图像、肢体行为等方式对其实施性骚扰。因此，小红受到性骚扰的行为是违法的。根据该法第二十四条，学校应当建立有效预防和科学处置性侵害、性骚扰的工作制度，并提供必要的保护措施。因此，小红可以向学校投诉，并要求学校采取相应的行动保护她的权益。受害妇女也有权向公安机关报案，或者向人民法院提起民事诉讼，以依法请求行为人承担民事责任。对于遭受性侵害、性骚扰的女学生，学校、公安机关、教育行政部门等相关单位和人员应当保护其隐私和个人信息，并提供必要的保护措施。因此，小红可以寻求这些相关单位的帮助和支持来维护自己的权益。
案例 2	小明和小红在城市郊区拥有各自的土地，小明希望借用小红的土地作为停车场，提高他自己土地的效益，双方应该怎样规划这个地役权合同？	根据《民法典》第三百七十二條，双方应订立书面地役权合同，明确以下条款：1. 双方身份信息；2. 供役地（小红土地）与需役地（小明土地）位置；3. 停车场使用范围及管理方式；4. 使用期限；5. 费用标准及支付方式；6. 争议解决条款。依据第三百七十四条，建议办理地役权登记以对抗第三人。	根据《民法典》第三百七十二條和第三百七十三條的规定，小明和小红可以通过订立地役权合同来规划他们之间的土地使用关系。地役权合同应当采用书面形式，并包括以下条款：（一）当事人的姓名或名称和住所；（二）供役地和需役地的位置；（三）利用目的和方法；（四）地役权期限；（五）费用及其支付方式；（六）解决争议的方法。小明作为需役权人，希望借用小红的土地作为停车场，提高自己土地的效益。因此，地役权合同中，小红应被确定为供役地的所有人，小明为需役地的所有人。合同应明确停车场的位置、使用目的和方法，以及合同的期限。双方还需就费用的支付方式和解决争议的方法达成一致。需要注意的是，根据《民法典》第三百七十四条的规定，地役权从合同生效时起设立。因此，当事人可以选择向登记机构申请地役权登记，以确保地役权的效力。若未经登记，地役权可能无法对抗善意第三人。综上所述，小明和小红可以通过订立一份书面的地役权合同，明确双方土地使用关系，以及其他相关条款，以保障双方的权益。同时，合同的登记可增加地役权的法律效力。

续下页

Table 4 (续)

ID	法律问题	模型回答	参考答案
案例 3	小明发现自己的信用评价并不正确，他可以采取哪些措施？	根据《民法典》第一千零二十九条，小明可向征信机构提出异议并要求更正。若涉及名誉侵害，依据第一千零二十八条可要求媒体删除不实信息。同时可援引第一千零三十条主张个人信息保护，必要时向网信部门投诉或提起诉讼。	根据《民法典》第一千零二十八条规定，若小明发现自己的信用评价失实并对其名誉权造成侵害，他有权请求媒体采取必要的更正或删除措施。此外，根据《民法典》第一千零二十九条规定，小明可以依法查询自己的信用评价，一旦发现不当评价，他有权提出异议并要求采取更正或删除等必要措施。信用评价人应负责核查，若属实，则应及时采取相应措施。另外，根据《民法典》第一千零三十条规定，小明与征信机构等信用信息处理者之间的关系适用个人信息保护的相关规定。最后，根据《民法典》第一千零三十一条规定，小明享有荣誉权，任何人不得非法剥夺其荣誉称号，不得诋毁或贬损其荣誉。如果小明的荣誉称号未记载，他可以请求记载；如果记载错误，他可以要求更正。综上所述，小明可以采取上述措施来应对他发现的信用评价不正确的情况。
案例 4	一起跨国犯罪的案件，办案机关需要获取在外国的相关人员的证言，应该如何申请国际刑事司法协助？	依据《国际刑事司法协助法》第二十五条，办案机关应：1. 制作附证据材料的请求书；2. 经最高人民法院或最高人民检察院审查；3. 通过司法部或外交部对外联系；3. 明确取证要求及法律依据；5. 可申请派员见证取证过程。	根据《国际刑事司法协助法》第二十五条的规定，办案机关需要获取在外国的相关人员的证言时，应按照以下程序申请国际刑事司法协助：1. 制作刑事司法协助请求书并附上相关材料；2. 由办案机关所属主管机关审核同意后，由对外联系机关及时向外国提出协助请求；3. 在协助请求书中清楚列明需要获取有关人员的证言；4. 同时可以请求外国在执行请求时派员到场。以上是根据相关法律条文得出的回答，每个结论都是根据法律条文和事实得出的。请注意回答要简洁明了，符合逻辑，且使用中文回答。
案例 5	某公司的一名职工提出申请，要求公司对其基本养老保险的缴纳情况进行公示。该职工提出的申请是否符合社会保险法相关规定？	根据《社会保险法》第四条和第六十条，职工有权监督单位缴费情况。依据第七十四条第三款，社保经办机构应定期公布社保情况。公司应当根据《企业信息公示暂行条例》第九条，公示社保缴纳信息，但需注意保护职工个人信息安全。	根据《社会保险法》相关规定，职工有权参与社会保险重大事项的研究，并对与职工社会保险权益有关的事项进行监督。根据第十条的规定，职工应当参加基本养老保险，由用人单位和职工共同缴纳基本养老保险费。基本养老保险实行社会统筹与个人账户相结合，基本养老保险基金由用人单位和个人缴费以及政府补贴等组成。根据职工提出的申请要求公司对其基本养老保险的缴纳情况进行公示，可以认定该申请符合《社会保险法》相关规定。职工作为参保人之一，有权了解并参与自己的养老保险缴费情况的公示和监督。

Key Observations and Discussion: This experiment successfully demonstrates the efficacy of fine-tuning the Qwen2.5-7B-Instruct model for legal domain adaptation using LoRA and 4-bit quantization. By leveraging the DISC-Law-SFT dataset and optimizing training parameters, the model achieved rapid convergence while maintaining computational efficiency on a T4 GPU. The evaluation revealed strong alignment with Chinese legal principles, with model responses exhibiting high factual accuracy and coherent structuring of legal arguments. Key observations include the effectiveness of LoRA in parameter-efficient adaptation and the necessity of early stopping to mitigate overfitting risks. While minor verbosity and phrasing discrepancies were noted, the results validate the viability of quantized LoRA for domain-specific LLM customization. Future work could explore full-parameter fine-tuning with extended training epochs to further enhance precision and reduce hallucination tendencies.



References and Acknowledgements

1. NLP From Scratch: Translation with a Sequence to Sequence Network and Attention: https://pytorch.org/tutorials/intermediate/seq2seq_translation_tutorial.html
2. BPE Tokenizer: <https://github.com/karpathy/minGPT/blob/master/minGPT/bpe.py#L257>
3. GPT2 Tokenizer: https://github.com/huggingface/transformers/blob/main/src/transformers/models/gpt2/tokenization_gpt2.py#L75
4. MinGPT: <https://github.com/karpathy/minGPT/>
5. DISC-Law-SFT Dataset: <https://huggingface.co/datasets/ShengbinYue/DISC-Law-SFT>
6. LLaMA-Factory: <https://github.com/hiyouga/LLaMA-Factory>
7. Qwen2.5-7B-Instruct: <https://huggingface.co/Qwen/Qwen2.5-7B-Instruct>